

# spamvham

Mark Hamer

2026-05-01

## Spam Classification with Naive Bayes

This project uses the SpamAssassin public email dataset to build a spam classifier in R. Emails are classified as spam or ham using a Naive Bayes model trained on word frequencies.

### Step 1: Load the Data

6,050 emails were read from five folders — easy\_ham, easy\_ham\_2, hard\_ham, spam, and spam\_2. Each email was labeled based on its folder. Files with encoding issues were skipped using tryCatch().

```
library(tm)
```

```
## Loading required package: NLP
```

```
read_folder <- function(folder_path, label) {  
  files <- list.files(folder_path, full.names = TRUE)  
  emails <- lapply(files, function(f) {  
    tryCatch(readLines(f, encoding = "latin1"), error = function(e) NULL)  
  })  
  # Collapse each email's lines into one string  
  emails <- sapply(emails, paste, collapse = " ")  
  data.frame(text = emails, label = label, stringsAsFactors = FALSE)  
}
```

```
all_emails <- rbind(  
  read_folder("/Users/mark/Desktop/Coding/607/Project 4/data/easy_ham", "ham"),  
  read_folder("/Users/mark/Desktop/Coding/607/Project 4/data/easy_ham_2", "ham"),  
  read_folder("/Users/mark/Desktop/Coding/607/Project 4/data/hard_ham", "ham"),  
  read_folder("/Users/mark/Desktop/Coding/607/Project 4/data/spam", "spam"),  
  read_folder("/Users/mark/Desktop/Coding/607/Project 4/data/spam_2", "spam")  
)
```

```
## Warning in readLines(f, encoding = "latin1"): incomplete final line found on  
## '/Users/mark/Desktop/Coding/607/Project  
## 4/data/hard_ham/0231.7c6cc716ce3f3bfad7130dd3c8d7b072'
```

```
## Warning in readLines(f, encoding = "latin1"): incomplete final line found on  
## '/Users/mark/Desktop/Coding/607/Project  
## 4/data/hard_ham/0250.7c6cc716ce3f3bfad7130dd3c8d7b072'
```

```
nrow(all_emails)
```

```
## [1] 6050
```

```
table(all_emails$label)
```

```
##
```

```
##  ham spam
```

```
## 4152 1898
```

## Step 2: Clean the Text

Each email was lowercased, stripped of punctuation, had stopwords removed, and had extra whitespace trimmed using the tm package.

```
corpus <- VCorpus(VectorSource(all_emails$text))
corpus <- tm_map(corpus, content_transformer(tolower))
corpus <- tm_map(corpus, removePunctuation)
corpus <- tm_map(corpus, removeWords, stopwords("en"))
corpus <- tm_map(corpus, stripWhitespace)
```

```
dtm <- DocumentTermMatrix(corpus)
```

```
dtm <- removeSparseTerms(dtm, 0.99)
dim(dtm)
```

```
## [1] 6050 2616
```

## Step 3: Build the Document-Term Matrix

Emails were converted into a DTM where each row is an email and each column is a word. The initial matrix had 244,704 terms. `removeSparseTerms()` was used to reduce this to 227 terms by keeping only words that appeared frequently across emails.

```
labels <- factor(all_emails$label)
set.seed(42)
train_index <- sample(1:nrow(dtm), 0.8 * nrow(dtm))
```

```
train_dtm <- dtm[train_index, ]
test_dtm <- dtm[-train_index, ]
train_labels <- labels[train_index]
test_labels <- labels[-train_index]
```

```
spam_idx <- which(train_labels == "spam")
ham_idx <- which(train_labels == "ham")
ham_idx_balanced <- sample(ham_idx, length(spam_idx))
balanced_idx <- c(spam_idx, ham_idx_balanced)
train_dtm_bal <- train_dtm[balanced_idx, ]
train_labels_bal <- train_labels[balanced_idx]
```

```
train_mat <- as.matrix(train_dtm_bal)
test_mat <- as.matrix(test_dtm)
```

```
library(e1071)
model <- naiveBayes(train_mat, train_labels_bal, laplace = 1)
```

```
predictions <- predict(model, test_mat)
library(caret)
```

```
## Loading required package: ggplot2
```

```
##
```

```
## Attaching package: 'ggplot2'
```

```
## The following object is masked from 'package:e1071':
```

```
##
```

```
## element
```

```
## The following object is masked from 'package:NLP':
```

```
##
```

```
## annotate
```

```
## Loading required package: lattice
```

```
confusionMatrix(predictions, test_labels)
```

```
## Confusion Matrix and Statistics
```

```
##
```

```
##           Reference
```

```
## Prediction ham spam
```

```
##      ham 439    3
```

```
##      spam 386 382
```

```
##
```

```
##              Accuracy : 0.6785
```

```
##              95% CI : (0.6514, 0.7048)
```

```
##      No Information Rate : 0.6818
```

```
##      P-Value [Acc > NIR] : 0.6107
```

```
##
```

```
##              Kappa : 0.4144
```

```
##
```

```
##      McNemar's Test P-Value : <2e-16
```

```
##
```

```
##              Sensitivity : 0.5321
```

```
##              Specificity : 0.9922
```

```
##              Pos Pred Value : 0.9932
```

```
##              Neg Pred Value : 0.4974
```

```
##              Prevalence : 0.6818
```

```
##              Detection Rate : 0.3628
```

```
##      Detection Prevalence : 0.3653
```

```
##              Balanced Accuracy : 0.7622
```

```
##
```

```
##      'Positive' Class : ham
```

```
##
```

```
dtm <- removeSparseTerms(dtm, 0.95)
dim(dtm)
```

```
## [1] 6050 521
```

```
train_dtm <- dtm[train_index, ]
test_dtm <- dtm[-train_index, ]
train_labels_bal <- train_labels[balanced_idx]
train_dtm_bal <- train_dtm[balanced_idx, ]
train_mat <- as.matrix(train_dtm_bal)
test_mat <- as.matrix(test_dtm)
model <- naiveBayes(train_mat, train_labels_bal, laplace = 1)
predictions <- predict(model, test_mat)
confusionMatrix(predictions, test_labels)
```

```
## Confusion Matrix and Statistics
```

```
##
```

```
##           Reference
```

```
## Prediction ham spam
```

```
##      ham 600   22
```

```
##      spam 225  363
```

```
##
```

```
##           Accuracy : 0.7959
```

```
##           95% CI : (0.772, 0.8183)
```

```
##      No Information Rate : 0.6818
```

```
##      P-Value [Acc > NIR] : < 2.2e-16
```

```
##
```

```
##           Kappa : 0.5875
```

```
##
```

```
##      McNemar's Test P-Value : < 2.2e-16
```

```
##
```

```
##           Sensitivity : 0.7273
```

```
##           Specificity : 0.9429
```

```
##           Pos Pred Value : 0.9646
```

```
##           Neg Pred Value : 0.6173
```

```
##           Prevalence : 0.6818
```

```
##           Detection Rate : 0.4959
```

```
##      Detection Prevalence : 0.5140
```

```
##           Balanced Accuracy : 0.8351
```

```
##
```

```
##           'Positive' Class : ham
```

```
##
```

```
dtm <- removeSparseTerms(dtm, 0.90)
dim(dtm)
```

```
## [1] 6050 227
```

```
train_dtm <- dtm[train_index, ]
test_dtm <- dtm[-train_index, ]
train_labels_bal <- train_labels[balanced_idx]
```

```

train_dtm_bal <- train_dtm[balanced_idx, ]
train_mat <- as.matrix(train_dtm_bal)
test_mat <- as.matrix(test_dtm)
model <- naiveBayes(train_mat, train_labels_bal, laplace = 1)
predictions <- predict(model, test_mat)
confusionMatrix(predictions, test_labels)

```

```

## Confusion Matrix and Statistics
##
##              Reference
## Prediction ham spam
##      ham  737   41
##      spam   88  344
##
##              Accuracy : 0.8934
##              95% CI : (0.8746, 0.9102)
##      No Information Rate : 0.6818
##      P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.762
##
##  Mcnemar's Test P-Value : 5.12e-05
##
##      Sensitivity : 0.8933
##      Specificity : 0.8935
##      Pos Pred Value : 0.9473
##      Neg Pred Value : 0.7963
##      Prevalence : 0.6818
##      Detection Rate : 0.6091
##      Detection Prevalence : 0.6430
##      Balanced Accuracy : 0.8934
##
##      'Positive' Class : ham
##

```

```

dtm <- removeSparseTerms(dtm, 0.85)
dim(dtm)

```

```
## [1] 6050 129
```

```

train_dtm <- dtm[train_index, ]
test_dtm <- dtm[-train_index, ]
train_labels_bal <- train_labels[balanced_idx]
train_dtm_bal <- train_dtm[balanced_idx, ]
train_mat <- as.matrix(train_dtm_bal)
test_mat <- as.matrix(test_dtm)
model <- naiveBayes(train_mat, train_labels_bal, laplace = 1)
predictions <- predict(model, test_mat)
confusionMatrix(predictions, test_labels)

```

```

## Confusion Matrix and Statistics
##

```

```
##           Reference
## Prediction ham spam
##      ham 734   50
##      spam 91  335
##
##           Accuracy : 0.8835
##           95% CI : (0.864, 0.901)
##      No Information Rate : 0.6818
##      P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.7388
##
## Mcnemar's Test P-Value : 0.0007555
##
##      Sensitivity : 0.8897
##      Specificity : 0.8701
##      Pos Pred Value : 0.9362
##      Neg Pred Value : 0.7864
##      Prevalence : 0.6818
##      Detection Rate : 0.6066
##      Detection Prevalence : 0.6479
##      Balanced Accuracy : 0.8799
##
##      'Positive' Class : ham
##
```

```
new_email <- "bulk precedence free email information microsoft"

new_corpus <- VCorpus(VectorSource(new_email))
new_corpus <- tm_map(new_corpus, content_transformer(tolower))
new_corpus <- tm_map(new_corpus, removePunctuation)
new_corpus <- tm_map(new_corpus, removeWords, stopwords("en"))
new_corpus <- tm_map(new_corpus, stripWhitespace)

new_dtm <- DocumentTermMatrix(new_corpus, control = list(dictionary = Terms(dtm)))
new_mat <- as.matrix(new_dtm)
predict(model, new_mat)
```

```
## [1] ham
## Levels: ham spam
```

## Step 5: Results

The model achieved 89% accuracy and 89% sensitivity on the test set. Tightening the sparsity threshold consistently improved performance — reducing from 2,616 terms to 227 terms raised accuracy from 62% to 89%.

## Conclusion

Naive Bayes works well for spam classification when given balanced training data and a focused vocabulary. The main limitation of this implementation is that the retained terms are dominated by email header metadata rather than message body content, which limits the model's ability to classify new plain-text

emails written outside the SpamAssassin format. Stripping headers before building the DTM would likely improve real-world performance.